



BSc (Hons) in **Computer Science**  
**SE3062 : Intelligent Systems**

Year 3 · Semester 1 · 2026 Semester 2

**Faculty of Computing**

## Practical 02

**Objective & Lecture Mapping:** In Lecture 03, we learned that a mathematically perfect "Table-Driven Agent" is impossible to build due to combinatorial explosion. Instead, we must write Agent Programs. Today, you will implement a **Simple Reflex Agent** using Condition-Action rules, observe its fatal flaws in a partially observable environment, and then upgrade it to a **Model-Based Agent** that maintains an internal memory state.

### Part 1: Practical Implementation — Coding the Architectures

#### Step 1.1: Setting the Trap (Partial Observability)

To see why Simple Reflex agents fail, we must handicap your agent's sensors. We are moving from a Fully Observable world to a Partially Observable one.

1. Open your `visual_grid_game.py` from Lab 01.
2. Locate the `get_percept(self)` method.
3. Modify it so it **no longer returns** the exact global coordinates ( `agent_pos` ). Instead, it should only return local booleans, e.g., `{'wall_ahead': True/False, 'food_here': True/False}` based on checking the adjacent cell in its current facing direction.

#### Step 1.2: The Simple Reflex Agent (Implementation & Failure)

1. Create a new class called `SimpleReflexAgent` .
2. Implement a `sense_and_act(self, percept)` method that uses strictly IF-THEN logic (Condition-Action rules). *Do not use an `__init__` method to store history.*

Example Logic: IF food\_here THEN suck; IF wall\_ahead THEN turn\_left; ELSE move\_forward

3. Run the simulation. **Observe:** Watch the agent get trapped in a corner or a U-shaped wall, infinitely repeating a cycle (e.g., turn left, step forward, hit wall, turn left...).

### Step 1.3: The Model-Based Agent (Memory & State)

1. Create a new class called `ModelBasedAgent`.
2. Implement an `__init__(self)` method to initialize an internal memory state (e.g., `self.visited_cells = set()` or a relative movement tracker).
3. In `sense_and_act(self, percept)`, first **update the state** (Transition & Sensor Model) by recording the current percept and the last action taken.
4. Update your IF-THEN rules to query the memory.  
Example Logic: IF wall\_ahead AND left\_is\_visited THEN turn\_right
5. Run the simulation. **Observe:** The agent should now remember where it has been, realize it is in a loop, and choose an alternate path to escape.

## Part 2: Theoretical Evaluation & Lecture Mapping

Based on your practical observations above, answer the following questions to map your code directly to the architectural theories covered in Lecture 02.

1. (Remember) According to Lecture 02, why is it impossible to program a mathematically perfect "Table-Driven Agent" for complex environments like Chess? What happens as the agent's lifetime increases?

A Table-Driven Agent would need to store an action for every possible sequence of percepts.  
As the agent's lifetime increases the number of possible percept sequences becomes extremely large.

2. (Understand) Look at the code you wrote for your `SimpleReflexAgent`. Identify and explain the specific lines of code that represent the "Condition-Action Rules" discussed in the lecture.

The SimpleReflexAgent uses simple IF-THEN rules to choose actions.  
For example, if food\_here is true then the agent performs the Right action.  
If wall\_ahead is true then it performs the Up action.  
If neither condition is true then it performs the default Right action.  
The agent only looks at the current percept and does not remember previous situations.

**3. (Analyze) Your SimpleReflexAgent likely got stuck in an infinite loop during Step 1.2. Based on the lecture, analyze exactly why this happened. How did the combination of "Partial Observability" and a lack of "Percept History" cause this failure?**

The Simple Reflex Agent can get stuck because it cannot see the complete environment and has no memory. Different locations can give the agent the same percept. When this happens the agent performs the same action again and again which can create a loop. A Model-Based Agent can avoid this by remembering previous situations and actions. The combination of "Partial Observability" and a lack of "Percept History" cause this failure as the agent could only see limited information about its surroundings and did not remember its previous percepts or actions, it could not know that it had already been in the same situation. Therefore it keeps making the same decisions and became stuck in an infinite loop.

**4. (Evaluate) In Step 1.3, you added an internal state to your ModelBasedAgent . Evaluate how your specific code handles the "Transition Model" (how the world evolves) and the "Sensor Model" (how the agent's actions affect the world).**

The Sensor Model is represented by get\_percept(). It tells the agent what it can currently see such as whether there is a wall ahead or food nearby.  
The Transition Model is represented by execute\_action(). It applies the agent's action and changes the environment such as changing the agent's position.  
The Model-Based Agent remembers previous percepts and actions allowing it to recognize repeated situations and choose a different action when necessary. This is how memory helps the agent deal with a partially observable environment.

Finalize and Lock Lab 02 Documentation

**Incomplete: 0/11 questions answered. Please provide detailed responses for all questions.**



**FACULTY OF COMPUTING**